

Blockchain-based Economic Voting with Posterior Security from Lattices

Navid Abapour¹ Amir Goharshady² Catalin Dragan¹

Mahdi Mahdavi³

¹Surrey Centre for Cyber Security, University of Surrey

`{n.abapour, c.dragan}@surrey.ac.uk`

²Department of Computer Science, University of Oxford

`amir.goharshady@cs.ox.ac.uk`

³K-ryptography and Information Security for Open Networks,

Universitat Oberta de Catalunya

`m_mahdavi@uoc.edu`

September 2025

Abstract

Electronic voting has demonstrated that it streamlines the democratic process, making it more convenient for citizens and enhancing the accuracy and speed of election results in real-world scenarios in the US, Estonia, Switzerland, and many other countries. One major challenge for e-voting, especially online voting, is ensuring that voting and tallying devices behave honestly, particularly in cases involving monetary transactions. These are addressed by economic voting, where everything is on-chain; in essence, voters utilize smart contracts to conduct all voting stages. There are very few results on economic voting, and none post-quantum secure. The challenge comes from having the entire voting system run by smart contracts. In this work, we propose the first post-quantum economic voting scheme, which combines hybrid on- and off-chain operations, called the Post-Quantum Blind Vote (PQBV). The core idea is to utilize smart contracts that enable blind signatures during the voting process. We enhance our contribution by introducing a post-quantum blind signature with Posterior Security, as proposed by Yuen et al. (CCS 2025), which retroactively enhances the privacy of already generated signatures. This has a significant impact on PQBV, as it is able to satisfy formal cryptographic privacy definitions, including ballot privacy. Our efficiency analysis reveals competitive

performance compared to existing state-of-the-art post-quantum e-voting systems, such as Epoque (EuroS&P 2021), which is done without blockchain.

Keywords: Electronic Voting, Lattice-based Cryptography, Economic Voting, Blind Signature, Posterior Security

1 Introduction

Electronic voting is considered an outstanding step towards modern democracies, as it enhances accessibility, security, and efficiency in elections, driven by a growing interest among citizens in integrating technology into government affairs [1], as they represent a firmly established and expanding component of electoral infrastructure, demonstrably employed for a spectrum of elections—from country-wide polls to local contests—in numerous countries, including, but not limited to, the US, Estonia, India, Switzerland, France, and Australia [2]. In some real-world financial scenarios where an election takes place (e.g., an auction), the voting process happens off-chain, and then the transaction happens separately on-chain using cryptocurrency. This separation in the process is inefficient, as it requires additional time and energy. This challenge can be addressed by utilizing e-voting schemes based on smart contracts, which are referred to as Economic Voting¹ [3]. For example, in an online casino or an auction [4], it is more efficient to conduct part/or all of the procedure on-chain because the entire transaction, including the money and voting, will be processed directly on the same route, and provides better transparency on the money flow [4, 5]. Additionally, it has been demonstrated that blockchain provides a reliable foundation for implementing a public bulletin board, offering a proactive approach to ensuring the integrity of published records by preventing retroactive modifications [6, 7].

However, not only are there a few promising results on blockchain-based voting, but also limitations occur as basing the voting system fully on-chain can fail to address core vulnerabilities like device compromise and network attacks [8] while introducing additional risks such as complexity, weak ballot secrecy, key management issues, scalability problems, and dependency on unproven technologies, and offer limited security benefits over simpler alternatives like traditional databases or paper ballots [9] and complicates governance and rapid vulnerability response. The existing economic approaches for voting are not cryptographically sufficient; for example, the Blind Vote lacks formal definitions of key terminology, such as secrecy and anonymity. Not only is this scheme not formalized, but it also lacks a formal proof of privacy and does not provide provable security guarantees [5]. It is based on classical computational assumptions, such as RSA, that, apart from its conventional vulnerabilities [10], is vulnerable to quantum attacks [11–13].

Despite the absence of post-quantum economic voting systems, from the post-quantum side, there has been substantial work on lattice-based non-economic e-

¹Economic voting, where voting processes are integrated with financial transactions on a blockchain, offers advantages in scenarios requiring both democratic decision-making and monetary flows. For instance, in Decentralized Autonomous Organizations (DAOs), stakeholders vote on funding proposals using governance tokens, where the voting outcome directly triggers on-chain fund allocation (see <https://ethereum.org/en/dao/>).

voting [14–16]. However, it is not straightforward to integrate a smart contract into this approach, as it requires compatible assumptions and primitives. For example, *Epoque* [14] is a significant result in post-quantum non-economic voting; but, *Epoque*’s utilization of multiple components, such as identity-based encryption with complex non-interactive zero-knowledge proofs, cannot be efficiently implemented or verified within the computational and gas constraints of current smart contract platforms.

In addition, as a recently presented concept, *Posterior Security* enables the enhancement of privacy properties for already-generated standard signatures without requiring access to the original signing keys [17], allowing applications to retroactively gain stronger anonymity and message hiding guarantees. This capability is particularly valuable in our blockchain voting context, as it permits the upgrading of deployed systems with enhanced privacy features while maintaining compatibility with existing signature infrastructure and enabling voters to obtain unlinkable ballots from standard blind signatures.

So, the problem is *how to achieve a posterior-secure voting scheme that can support economic operations and be securely used in the post-quantum era?*

Contributions. In this work, we use the potential of blockchain to design post-quantum economic voting with lattices, then we embed this variant into an efficient structure of Blind Vote [5], along with achieving ballot privacy. Our contributions include:

- presenting the first provable ballot-private voting system based on smart contract; a step towards bridging the gap between cryptographers and the cryptocurrency communities’ idea on blockchain-based voting.
- constructing a Post-Quantum version of Blind Vote (PQBV) using lattices.
- demonstrating the first formalization and application of *Posterior Security* in blockchain, enabling anonymity and message hiding for an already generated standard signature, even by someone who has no access to the signing key.

Organization. Section 3 establishes the notation and building blocks necessary for our construction. Section 4 introduces the concept of posterior security and formalizes our blind signature scheme. Section 5 presents the complete construction of our post-quantum blockchain voting system PQBV, detailing the integration of functions with smart contracts. Section 6 presents a formal security proof that demonstrates our scheme achieves ballot privacy and unforgeability, while also analyzing its computational complexity and comparing our scheme’s performance with *Epoque* [14]. Finally, Section 7 summarizes our contributions and discusses future research directions.

2 Related Works

To the best of our knowledge, no post-quantum economic voting scheme has been presented that has been proven to be ballot-private. Therefore, in this section, we highlight the need for such a scheme by examining the most similar schemes to ours, as presented from both cryptographic and cryptocurrency perspectives.

2.1 The Existing Lattice-based E-voting Schemes

To highlight the gap mentioned in the previous section, due to not considering economic operations and inheriting a high computational cost, these lattice-based schemes [14–16] cannot easily be transformed into variants with blockchain.

2.1.1 Epoque [14]

In this system, each voter V_i encodes their vote as $\mathbf{v}^i = (v^{i,j})_{j=1}^{n_{\text{cand}}} \in \{0,1\}^{n_{\text{cand}}}$ and secret shares each component among n_T trustees as $v^{i,j} = (v_1^{i,j}, \dots, v_{n_T}^{i,j})$, then commits to each share using a homomorphic commitment scheme $c_k^{i,j} \leftarrow \text{Com}(\text{prm}_{\text{com}}, v_k^{i,j}; r_k^{i,j})$ and encrypts the opening values under trustee T_k 's IBE master public key using identity i : $e_k^i \leftarrow \text{Enc}(\text{prm}_k, i; (v_k^{i,j}, r_k^{i,j})_{j=1}^{n_{\text{cand}}})$. During tallying, each trustee T_k uses master shortcut decryption to verify ballot validity and publishes $v_k^j \leftarrow \sum_{i=1}^{n_V} v_k^{i,j}$ and $r_k^j \leftarrow \sum_{i=1}^{n_V} r_k^{i,j}$ for each candidate j , with correctness verified via $\text{Open}(\text{prm}_{\text{com}}, v_k^j, c_k^j, r_k^j) = 1$ where $c_k^j \leftarrow \sum_{i=1}^{n_V} c_k^{i,j}$ using the homomorphic property. If a trustee claims a ballot is invalid, they must publish the voter's individual IBE secret key $\text{msk}_k^i \leftarrow \text{Extr}(\text{prm}_k, \text{msk}_k, i)$ to prove the claim.

2.1.2 Lattice-Based Electronic Voting from NTRU [15]

This scheme works using NTRU encryption combined with threshold blind signatures, where each voter V_i encrypts their vote $v \in \mathcal{R}_p$ as $c = p(hs + e) + v \in \mathcal{R}_q$ using NTRU public key $h = g/f$ and encryption randomness $(s, e) \in \mathcal{S}_v^2$, then obtains a threshold blind signature σ from at least t signing authorities on the ciphertext c . The ballot $b = (c, \sigma)$ undergoes verifiable shuffling through ξ_1 mix servers using NTRU-based mix-nets with zero-knowledge proofs π_{Small} and π_{Shuf} to prove correct re-randomization and permutation, followed by threshold distributed decryption where each of ξ_2 decryption servers computes shares $ds_{i,j} = dk_j \cdot c_i + p \cdot E_{i,j}$ with noise drowning $E_{i,j} \leftarrow \mathcal{S}_{B_{\text{Drown}}}$ and provides exact zero-knowledge proofs π_{Lin} and π_{Bnd} of correct computation and boundedness. Finally, the votes are recovered by combining decryption shares as $v_i = (\sum_{j \in [\xi_2]} ds_{i,j} \bmod q) \bmod p$.

2.1.3 Post-quantum E-voting Scheme from Ring-LWE [16]

The voting protocol operates in three phases: first, each voter V_i encrypts their vote $v \in \{0,1\}^*$ using a Ring-LWE-based encryption scheme to produce ciphertext $c = \text{PK.Encrypt}(v, mk)$ where mk is the public election key. Second, the voter obtains a threshold blind signature $\sigma = \text{BS.Sign}(\{A_j([sk]_j)\}_{j \in T}, V_i(pk, c))$ from at least t signing authorities on the ciphertext c , where the blind signature scheme is based on lattice trapdoors and provides perfect blindness. Finally, the ballot $b = (c, \sigma)$ is posted to the bulletin board, and after the election closes, at least t authorities reveal their decryption key shares $[ek]_j$ to reconstruct the full decryption key ek , allowing anyone to verify signatures and decrypt ballots to compute the election result $r = \{v_i : \text{PK.Decrypt}(c_i, ek) = v_i \text{ for valid } (c_i, \sigma_i) \in BB\}$.

2.2 Economic E-voting Systems

These blockchain-based schemes [3, 5] lack long-term privacy and a post-quantum level of security, and their structures make it difficult to achieve posterior security with them. What makes it even harder is that putting lattice structures on-chain can consume a huge amount of gas on the smart contract, so a hybrid approach can be more effective, which we will present in Section 5 how we did it.

2.2.1 Tornado Vote [3]

This system combines Tornado Cash cryptocurrency mixing with zero-knowledge proofs, where voters deposit ERC-20 voting tokens into a vault with Pedersen hash $H_{Ped}(sect||k)$ (where $sect$ is a random secret and k is a nullifier), then commit to their vote v by sending the first 20 bytes of $H_{SHA}(sec_c||v)$ along with $H_{Ped}(k)$ and a ZK proof of secret knowledge to achieve anonymity before voting. During the reveal phase, voters disclose v and sec_c through relayers to the smart contract, which verifies $H_{SHA}(sec_c||v)$ against stored commitments and transfers voting tokens to unowned addresses representing each vote choice, with nullifiers preventing double voting and final tallies determined by token balances at outcome addresses.

2.2.2 Blind Vote [5]

In this scheme, voters generate RSA key pairs (N_i, e_i, d_i) and obtain blind signatures from the administrator on their public key hash $h_i = \text{hash}(N_i, e_i)$ through the protocol $h'_i = h_i \cdot r_i^e \bmod N \rightarrow s'_i = (h'_i)^d \bmod N \rightarrow s_i = s'_i \cdot r_i^{-1}$, then commit to their vote by submitting $(N_i, e_i, s_i, c_i, sc_i)$ where $c_i = \text{hash}(v_i, x_i)$ and $sc_i^{e_i} = c_i \bmod N_i$ proves knowledge of the vote commitment. During the reveal phase, voters disclose (c_i, v_i, x_i) through relays for contract verification of $\text{hash}(v_i, x_i) = c_i$, enabling anonymous vote tallying while maintaining complete blockchain verifiability through the blind signature protocol that decouples voter identity from vote content.

3 Preliminaries

Let $\lambda \in \mathbb{R}$ denote the security parameter. Given positive integers $m, n \in \mathbb{N}$, we define $[m] := \{1, 2, \dots, m\}$ and $[n, m] := \{n, n+1, \dots, m\}$. We denote by $\mathbb{Z}_m$ the quotient ring of integers modulo m , where elements are represented using the interval $[-m/2, m/2) \cap \mathbb{Z}$, and by $\mathbb{Z}_m^*$ its group of units. For any vector $\mathbf{x} \in \mathbb{R}^m$, the notation $\|\mathbf{x}\|$ represents the Euclidean norm $\|\mathbf{x}\|_2$, while $\|\mathbf{x}\|_\infty$ denotes the maximum norm.

3.1 Computational Assumptions

We recall a set of hard mathematical problems required by the security proof for our construction.

Randomized One-More Inhomogeneous Short Integer Solution (rOM-ISIS) [18]

Let $n, m, q \in \mathbb{N}$ and let $\beta > 0$. In the random oracle model, given $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$ and access to a random oracle $\mathcal{H}: \{0,1\}^* \rightarrow \mathbb{Z}_q^n$, the $\text{rOM-ISIS}(n, m, q, \beta)$ problem asks an adversary to output $\ell + 1$ pairs $\{(\mathbf{z}_i, \mathbf{t}_i)\}_{i=0}^\ell$ such that $\mathbf{A}\mathbf{z}_i = \mathbf{t}_i \pmod{q}$, $\|\mathbf{z}_i\| \leq \beta$, and $\mathbf{z}_i \neq \mathbf{0}$, after making at most ℓ queries to $\mathcal{H}$.

Module Short Integer Solution (MSIS) [19]

Let $n, k, m \in \mathbb{N}$, $q \in \mathbb{N}$, and let $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. Let $\beta > 0$. Define $\text{MSIS}(n, k, m, q, \beta)$ as the problem of finding a non-zero vector $\mathbf{z} \in R_q^m$ such that $\mathbf{A}\mathbf{z} = \mathbf{0} \pmod{q}$ and $\|\mathbf{z}\|_\infty \leq \beta$, where $\mathbf{A} \leftarrow R_q^{k \times m}$.

Module Learning with Errors (MLWE) [19]

Let $n, k, m \in \mathbb{N}$, $q \in \mathbb{N}$, and let $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. Let χ be a distribution over R_q . Consider $D_0 := (\mathbf{A}, \mathbf{A}^T \mathbf{s} + \mathbf{e})$ and $D_1 := (\mathbf{A}, \mathbf{u})$, where $\mathbf{A} \leftarrow R_q^{k \times m}$, $\mathbf{s} \leftarrow R_q^k$, $\mathbf{e} \leftarrow \chi^m$, $\mathbf{u} \leftarrow R_q^m$. The $\text{MLWE}(n, k, m, q, \chi)$ problem is to decide between the distributions D_0 and D_1 .

3.2 Building Blocks

Non-Interactive Blind Signature (NIBS_{rOM}) [20]

This signature, which is based on Randomized One-More Inhomogeneous Short Integer Solution, utilizes a CPA-secure public key encryption scheme² $\text{PKE} = (\text{PKE.KeyGen}, \text{PKE.Enc}, \text{PKE.Dec})$, and a non-interactive zero-knowledge proof $\text{NIZK} = (\text{NIZK.Setup}, \text{NIZK.Prove}, \text{NIZK.Verify})$ for linear relations over $\mathbb{Z}_q$, lattice trapdoors $\text{bLT} = (\text{bLT.TrapGen}, \text{bLT.SamplePre})$, and hash function $H: \{0,1\}^* \rightarrow \mathbb{Z}_q^n$.

Definition 1. For parameters $n = \text{poly}(\lambda)$, $m > n \log q + \lambda$, prime q , Gaussian parameter of $\varsigma = \Omega(m)$, and norm bound as $\beta = \varsigma \sqrt{m}$, the scheme $\text{NIBS}_{\text{rOM}} = (\text{Setup}, \text{KeyGen}_S, \text{KeyGen}_R, \text{Issue}, \text{Obtain}, \text{Verify})$ operates as:

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: Outputs the public parameters $\text{pp} = (\mathbf{A}, \mathbf{B}, \text{pke.pk}, \text{NIZK.crs})$ where $\mathbf{A}, \mathbf{B} \xleftarrow{R} \mathbb{Z}_q^{n \times 2m}$, $(\text{pke.pk}, \cdot) \leftarrow \text{PKE.KeyGen}(1^\lambda)$, and the setup of $\text{NIZK.crs} \leftarrow \text{NIZK.Setup}(1^\lambda)$ for language $\mathcal{L}_1 = \{(\mathbf{C}, \mathbf{A}, \mathbf{B}, \text{pke.pk}, \text{ct}, \mathbf{w}, \delta) : \exists(\mathbf{x}, \mathbf{y}, \mathbf{z}, r) \text{ satisfying the relation}\}$.
- $(\text{sk}, \text{vk}) \leftarrow \text{KeyGen}_S(\text{pp})$: Gives secret key $\text{sk} = \mathbf{T}_C$, $\text{vk} = \mathbf{C}$ where $(\mathbf{T}_C, \mathbf{C}) \leftarrow \text{bLT.TrapGen}(1^\lambda, n, 2m, q)$.
- $(\text{sk}_R, \text{pk}_R) \leftarrow \text{KeyGen}_R(\text{pp})$: Outputs $\text{sk}_R = (\mathbf{x}, \delta)$, $\text{pk}_R = \mathbf{A} \cdot \mathbf{x} + H(\delta)$ where $\mathbf{x} \leftarrow D_{\mathbb{Z}^{2m}, \varsigma/m}$, $\delta \xleftarrow{R} \{0,1\}^\lambda$.

²The choice of IND-CPA security (rather than CCA) is sufficient because the PKE is only used to encrypt the witness for the ZKP proof, and the ZKP provides the necessary non-malleability guarantees.

- $(\text{psig}, \text{nonce}) \leftarrow \text{Issue}(\text{sk}, \text{pk}_R)$: For $\mathbf{y} \xleftarrow{R} \{-1, +1\}^{2m}$, outputs $\text{psig} = \mathbf{z}$, $\text{nonce} = \mathbf{y}$ where $\mathbf{z} \leftarrow \text{bLT.SamplePre}(\mathbf{C}, \mathbf{T}_C, \text{pk}_R - \mathbf{B} \cdot \mathbf{y}, \varsigma)$.
- $(\mu, \sigma) \leftarrow \text{Obtain}(\text{sk}_R, \text{vk}, \text{psig}, \text{nonce})$: Verifies validity conditions. If valid, outputs $\mu = (\mathbf{w}, \delta)$, $\sigma = (\pi, \text{ct})$ where $\mathbf{w} = \mathbf{A} \cdot [\mathbf{x}_\perp^T \| \mathbf{z}_\perp^T]^T$, and the ciphertext $\text{ct} \leftarrow \text{PKE.Enc}(\text{pke}, \text{pk}, \mathbf{x} \| \mathbf{y} \| \mathbf{z})$, and π is the NIZK proof for instance $(\mathbf{C}, \mathbf{A}, \mathbf{B}, \text{pke}, \text{pk}, \text{ct}, \mathbf{w}, \delta) \in \mathcal{L}_1$.
- $b \leftarrow \text{Verify}(\text{vk}, \mu, \sigma)$: This sub-procedure outputs the result of verification by using zero-knowledge $\text{NIZK.Verify}(\text{NIZK.crs}, (\mathbf{C}, \mathbf{A}, \mathbf{B}, \text{pke}, \text{pk}, \text{ct}, \mathbf{w}, \delta), \pi)$.

The scheme achieves one-more-unforgeability under the $\text{rOM-ISIS}_{q,n,2m,\varsigma,3\sqrt{2}\beta}$ and receiver blindness under ZKP zero-knowledge and PKE semantic security.

Polynomial Commitment (PC)

The scheme from [21], based on Module Short Integer Solution, enables a prover to commit to a polynomial $h(X) \in \mathbb{Z}_p[X]$ and later prove evaluations $h(x) = y$ with square-root-sized proofs. The construction utilizes a modified Ajtai commitment [22] $\text{Com} = (\text{Setup}, \text{Com}, \text{Open})$ with randomized encoding $\text{R.Ecd} : \mathbb{Z}_p^{d/r} \times \mathbb{R}_{>0} \rightarrow R$ mapping large prime field elements to small-coefficient ring elements, where $R = \mathbb{Z}[X]/(X^d + 1)$ and $p = b^r + 1$.

Definition 2. For polynomial degree bound $N = nm$ and security parameter λ , the polynomial commitment scheme $\text{PC} = (\text{Setup}, \text{Com}, \text{Open}, \text{Eval}, \text{Verify})$ consists of:

- $\text{ck} \leftarrow \text{Setup}(1^\lambda, N)$: Outputs commitment key $\text{ck} = (A_0, A_1)$ where $A_0 \leftarrow \mathcal{U}(R_q^{\mu \times \ell})$, $A_1 = [A'_1 | I_\mu] \in R_q^{\mu \times (\mu + \nu)}$ with $A'_1 \leftarrow \mathcal{U}(R_q^{\mu \times \nu})$, for $n = d\ell/r$.
- $(\vec{h}, \delta) \leftarrow \text{Com}(\text{ck}, h(X))$: For $h(X) = \sum_{i=0}^{N-1} h_i X^i \in \mathbb{Z}_p[X]$, outputs commitment $\vec{h} = \vec{h}_0 \| \dots \| \vec{h}_{m+1} \in R_q^{\mu(m+2)}$ and opening $\delta = (\vec{h}, \vec{\eta})$ where:
 - $\vec{h}_i \leftarrow \text{R.Ecd}(\vec{h}_i; s_1)$, $\vec{\eta}_i \leftarrow D_{\mathbb{Z}_d}^{\mu + \nu, \sigma_1}$, $\vec{h}_i = A_0 \vec{h}_i + A_1 \vec{\eta}_i \pmod{q}$ for $i \leq m$
 - $\vec{h}_{m+1} \leftarrow \text{R.Ecd}(\vec{h}_{m+1}; \sqrt{m+2} \cdot s_3)$, $\vec{\eta}_{m+1} \leftarrow D_{\mathbb{Z}_d}^{\mu + \nu, \sqrt{m+2} \cdot \sigma_3}$
- $b \leftarrow \text{Open}(\text{ck}, \vec{h}, h(X), \delta)$: Outputs 1 iff $\|2\vec{h}_i\|_2 \|2\vec{\eta}_i\|_2 \leq 2d\beta_{\text{Open}}$ for all i and polynomial reconstruction holds.
- $(y, \rho) \leftarrow \text{Eval}(x, \delta)$: Computes $\vec{e} = \sum_{i=0}^{m-1} \text{Ecd}(x^{n_i}) \cdot \vec{h}_i + \text{Ecd}(x) \cdot \vec{h}_m + \vec{h}_{m+1}$, outputs $y = \langle \text{Dcd}(\vec{e}), (1, x, \dots, x^{n-1}) \rangle \pmod{p}$ and proof $\rho = (\vec{e}, \vec{\epsilon})$.
- $b \leftarrow \text{Verify}(\text{ck}, \vec{h}, x, y, \rho)$: Accepts iff $\|\vec{e}\|_2 \leq \beta_{\text{Eval}}$ and consistency checks pass.

The scheme satisfies computational hiding under $\text{MLWE}_{R, \nu, q, \sigma_1}$, binding under $\text{MSIS}_{R, \mu, q, 4\beta_{\text{PC}}}$, and evaluation binding under $\text{MSIS}_{R, \mu, q, 2\beta_{\text{Eval}}}$, where $\beta_{\text{PC}} = \beta_{\text{Eval}} + \frac{(b+1)(m+1)dr}{2} \cdot \beta_{\text{Open}}$.

Blockchain Operations

We model the general blockchain as a system $\text{Blockchain} = (\text{Deploy}, \text{Submit}, \text{GetDecoys})$ with the following operations:

- $\text{contract} \leftarrow \text{Deploy}(\text{code}, \text{params})$: Deploys smart contract code with parameters to the blockchain, returning contract address.
- $\text{tx} \leftarrow \text{Submit}(\text{contract.function}(\text{args}), \text{value})$: Submits a transaction calling contract function with arguments and optional payment value.
- $\text{keys} \leftarrow \text{GetDecoys}(\text{contract}, k)$: Retrieves k dummy verification keys from the blockchain to create an anonymity set that will hide the actual signer’s verification key during the posterior security conversion.

3.3 Ballot Privacy

Proving a voting scheme secure against ballot privacy (BPRIV) means that ballots do not leak information on votes. As formalized in Definition 3 by [23], it should be impossible, even for active adversaries who can submit arbitrary ballots.

Definition 3. *A voting scheme $\mathcal{V}$ has ballot privacy if there exists a simulator Sim such that no efficient adversary $\mathcal{A}$ can distinguish between the games $\text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}, I}^{\text{BPRIV}, 0}(\lambda)$ and $\text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}, I}^{\text{BPRIV}, 1}(\lambda)$ defined in Figure 1. That is, the expression*

$$\left| \Pr \left[\text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}, I}^{\text{BPRIV}, 0}(\lambda) = 1 \right] - \Pr \left[\text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}, I}^{\text{BPRIV}, 1}(\lambda) = 1 \right] \right|$$

is negligible in λ , for any set of voters I .

An experiment is shown in Figure 1; in the experiments $\text{Exp}_{\mathcal{A}, \mathcal{V}, I}^{\text{bpriv}, \beta}$, the adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ has access to the set of oracles $\mathcal{O} = \{\text{Ocast}, \text{Ovote}, \text{Otally}, \text{Oboard}\}$. The adversary is allowed to call the *Otally* oracle at most once. In this model, the adversary tries to distinguish between two worlds.

4 Intuition on Posterior Security

4.1 Syntax and Security Properties

Posterior Security addresses a gap by enabling stronger privacy features to be applied to standard signatures after they are generated, allowing applications like two-tier Central Bank Digital Currencies [17] to gain further post-hoc privacy guarantees without any changes to existing signing algorithms. From [17], in Definition 4, we formalize the concept of posterior security³.

Definition 4. *Given a signature scheme $\Sigma = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$, a posterior security transformation $\Pi = (\text{PS.Setup}, \text{PS.Convert}, \text{PS.CVerify})$ enhances signatures with additional security properties post-generation, where*

³It provides two properties: posterior anonymity (via incognito signature) and message hiding (via concealed signature). For additional information, refer to Appendix A.2.

$\text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}, I}^{\text{bpriv}, \beta}(\lambda)$	$\mathcal{O}_{\text{tally}}()$ for $\beta=1$
1: $\text{BB}_0, \text{BB}_1 \leftarrow []$ 2: $\text{cL}, \text{uL} \leftarrow \text{empty}$ 3: $(\text{pk}, \text{sk}) \leftarrow \text{Setup}(1^\lambda)$ 4: $\forall id. id \in I \text{ do } \text{uL}[id] \leftarrow \text{Register}(id)$ 5: $L \leftarrow \mathcal{A}_1(I)$ 6: $\text{coL} \leftarrow \{id \mid id \in I \wedge id \in L\}$ 7: $\forall id. id \in \text{coL} \text{ do } \text{cL}[id] \leftarrow \text{uL}[id]$ 8: $\beta' \leftarrow \mathcal{A}_2^{\mathcal{O}}(\text{pk}, \text{cL})$ 9: return β'	1: $(r, \Pi) \leftarrow \text{Tally}(\text{BB}_0, \text{sk})$ 2: $\Pi' \leftarrow \text{Sim}(\text{pk}, \text{Publish}(\text{BB}_1), r)$ 3: return (r, Π')
$\mathcal{O}_{\text{cast}}(id, b)$	$\mathcal{O}_{\text{vote}}(id, v_0, v_1)$
1: if $(id \in \text{coL} \wedge \text{Valid}(\text{BB}_\beta, b, \text{pk}))$ then 2: $\text{BB}_0 \leftarrow \text{BB}_0 + [b]; \text{BB}_1 \leftarrow \text{BB}_1 + [b]$	1: $(\text{upk}, \text{usk}) \leftarrow \text{uL}[id]$ 2: if $(id \in I \wedge id \notin \text{coL})$ then 3: $b_0 \leftarrow \text{Vote}(id, v_0, \text{pk}, \text{usk})$ 4: $b_1 \leftarrow \text{Vote}(id, v_1, \text{pk}, \text{usk})$ 5: if $(\text{Valid}(\text{BB}_\beta, b_\beta, \text{pk}))$ then 6: $\text{BB}_0 \leftarrow \text{BB}_0 + [b_0]$ 7: $\text{BB}_1 \leftarrow \text{BB}_1 + [b_1]$
$\mathcal{O}_{\text{tally}}()$ for $\beta=0$	$\mathcal{O}_{\text{board}}()$
1: $(r, \Pi) \leftarrow \text{Tally}(\text{BB}_0, \text{sk})$ 2: return (r, Π)	1: return $\text{Publish}(\text{BB}_\beta)$

Figure 1: Ballot Privacy Experiment and Oracles [23]

- $\text{PS.Setup}(1^\lambda, \text{param}) \rightarrow \text{param}_{\text{PS}}$
- $\text{PS.Convert}(\sigma, m, \text{pk}^*, \text{aux}) \rightarrow \sigma_{\text{PS}} \cup \{\perp\}$ *outputs* $\perp \Leftrightarrow \text{Verify}(\sigma, m, \text{pk}^*) = 0$
- $\text{PS.CVerify}(\sigma_{\text{PS}}, \text{stmt}, \text{aux}) \rightarrow \{0, 1\}$

have correctness: $\forall (\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(), \sigma \leftarrow \text{Sign}(m, \text{sk})$:

$$\text{PS.CVerify}(\text{PS.Convert}(\sigma, m, \text{pk}, \text{aux}), \text{stmt}, \text{aux}) = 1.$$

Posterior Anonymity.

Let the Incognito Signature to be $\Pi_{\text{IS}} = (\text{IS.Setup}, \text{IS.Convert}, \text{IS.CVerify})$ where $\text{aux} = \text{pk} = \{\text{pk}_1, \dots, \text{pk}_n\}$, $\text{pk}^* \in \text{pk}$, and $\text{stmt} = m$. We define $\text{Exp}_{\mathcal{A}, \Pi_{\text{IS}}}^{\text{s-anon}}(\lambda)$ by:

$$\begin{cases} (\text{sk}_i, \text{pk}_i)_{i \in [n]} \leftarrow \text{KeyGen}(\omega_i); \text{param}_I \leftarrow \text{IS.Setup}() \\ (m^*, i_0, i_1, \text{pk}^*, \sigma_0^*, \sigma_1^*) \leftarrow \mathcal{A}_1(\text{param}_I, \{\text{pk}_i, \omega_i\}_{i \in [n]}) \\ b \xleftarrow{\$} \{0, 1\}; \sigma_I^* \leftarrow \text{IS.Convert}(\sigma_b^*, m^*, \text{pk}_{i_b}, \text{pk}^*) \\ b' \leftarrow \mathcal{A}_2(\sigma_I^*); \text{return } b' \stackrel{?}{=} b \end{cases}$$

where $\text{pk}_{i_0}, \text{pk}_{i_1} \in \text{pk}^*$ and $\sigma_j^* = \text{Sign}(m^*, \text{sk}_{i_j})$. For the adversary's advantage in strong anonymity we have $\text{Adv}_{\mathcal{A}}^{\text{s-anon}} := |\Pr[\text{Exp}_{\mathcal{A}}^{\text{s-anon}} = 1] - 1/2| \leq \text{negl}(\lambda)$.

Posterior Message Hiding.

Let the Concealed Signature to be $\Pi_{CS} = (\text{CS.Setup}, \text{CS.Convert}, \text{CS.CVerify}, \text{CS.Decom})$ where $\text{stmt} = \text{pk}$. We define $\text{Exp}_{\mathcal{A}, \Pi_{CS}}^{\text{s-hide}}(\lambda)$ as:

$$\begin{cases} (\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(); \text{param}_{CS} \leftarrow \text{CS.Setup}() \\ (m_0^*, m_1^*, \sigma_0^*, \sigma_1^*) \leftarrow \mathcal{A}_1(\text{param}_{CS}, \text{pk}, \text{sk}) \\ b \xleftarrow{\$} \{0, 1\}; (\sigma_{CS}^*, \text{aux}^*) \leftarrow \text{CS.Convert}(\sigma_b^*, m_b^*, \text{pk}) \\ b' \leftarrow \mathcal{A}_2(\sigma_{CS}^*); \text{return } b' \stackrel{?}{=} b \end{cases}$$

where $\sigma_j^* = \text{Sign}(m_j^*, \text{sk})$. For the adversary's advantage in strong hiding we have: $\text{Adv}_{\mathcal{A}}^{\text{s-hide}} \leq \text{negl}(\lambda)$.

Unforgeability.

For $\text{Exp}_{\mathcal{A}}^{\text{forge}}(\lambda)$, adversary $\mathcal{A}$ has oracle access to $\mathcal{O}_{\text{sign}}(m, \text{pk}_i)$, $\mathcal{O}_{\text{corr}}(\text{pk}_i)$, and $\mathcal{O}_{\text{conv}}(m, \text{pk}', \text{pk}_i)$. $\mathcal{A}$ wins if it outputs $(\text{pk}^*, \sigma^*, m^*)$ where the verification function $\text{PS.CVerify}(\sigma^*, \cdot, \text{pk}^*) = 1$ and $\forall \text{pk}_i \in \text{pk}^*$: no queries (m^*, pk_i) to $\mathcal{O}_{\text{sign}}$, pk_i to $\mathcal{O}_{\text{corr}}$, or $(m^*, \text{pk}^*, \text{pk}_i)$ to $\mathcal{O}_{\text{conv}}$, then $\Pr[\text{Exp}^{\text{forge}} = 1] \leq \text{negl}(\lambda)$.

4.2 Posterior-Secure Blind Signature

We transform the lattice-based scheme NIBS_{rOM} into a posterior-secure variant $\Pi_{\text{PS-NIBS}}$ by augmenting it with posterior security transformations. Given $\text{NIBS}_{\text{rOM}} = (\text{Setup}, \text{KeyGen}_S, \text{KeyGen}_R, \text{Issue}, \text{Obtain}, \text{Verify})$ remain unchanged (inherited from NIBS_{rOM}), we construct $\Pi_{\text{PS-NIBS}}$ with additional algorithms, where first the base components get considered as a commitment scheme $\text{COM} = (\text{Com}, \text{Decom})$ with binding and hiding properties, and an extended ZKP system for languages $\mathcal{L}_{\text{anon}}$ and $\mathcal{L}_{\text{hide}}$ defined in the Figure 2, Figure 6, and Figure 7.

Therefore, $\Pi_{\text{PS-NIBS}} = (\text{Setup}, \text{KeyGen}_S, \text{KeyGen}_R, \text{Issue}, \text{Obtain}, \text{Verify}, \text{PS.Setup}, \text{PS.Convert}, \text{PS.CVerify})$ where PS.Convert and PS.CVerify internally uses the IS/CS algorithms based on the chosen mode.

Theorem 1 investigates the posterior security of $\Pi_{\text{PS-NIBS}}$, and establishes that our lattice-based construction maintains all essential security properties from the original blind signature scheme while adding the new posterior security capabilities.

Theorem 1. *The construction $\Pi_{\text{PS-NIBS}}$ satisfies posterior security (Definition 4) under the following assumptions:*

1. *Posterior anonymity under ZKP zero-knowledge and PKE semantic security.*
2. *Strong posterior message hiding under ZKP zero-knowledge, PKE semantic security, and commitment hiding.*
3. *Unforgeability under rOM-ISIS, soundness of ZKP, and commitment binding.*

Proof. We prove each property separately. Given adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ against $\text{Exp}^{\text{s-anon}}$, we construct a sequence of hybrids:

PS.Setup($1^\lambda, pp$)	PS.Convert($\sigma, \mu, mode, aux$)
1: $(A, B, pke.pk, nizek.crs) \leftarrow pp$	1: if $mode = anon$:
2: $nizek.crs_{anon} \leftarrow NIZK.Setup(1^\lambda, \mathcal{L}_{anon})$	2: return IS.Convert(σ, μ, aux)
3: $nizek.crs_{hide} \leftarrow NIZK.Setup(1^\lambda, \mathcal{L}_{hide})$	3: if $mode = hide$:
4: $pp_{PS} := (pp, nizek.crs_{anon}, nizek.crs_{hide})$	4: return CS.Convert(σ, μ, aux)
5: return pp_{PS}	

PS.CVerify($\sigma_{PS}, stmt, mode, aux$)
1: if $mode = anon$:
2: return IS.CVerify($\sigma_{PS}, stmt, aux$)
3: if $mode = hide$:
4: return CS.CVerify($\sigma_{PS}, stmt, aux$)

Figure 2: Posterior Security Algorithms

- Hyb₀: Real experiment Exp^{s-anon} with $b=0$
- Hyb₁: Replace π_I with simulated proof of ZKP
- Hyb₂: Substitute ct_I with encryption of $0^{|I|}$
- Hyb₃: As Hyb₂ but with $b=1$

By zero-knowledge property of ZKP we have $|\Pr[\text{Hyb}_0 = 1] - \Pr[\text{Hyb}_1 = 1]| \leq \text{Adv}_{\text{NIZK}}^{\text{zk}}(\lambda)$ and by semantic security property of the PKE scheme we have $|\Pr[\text{Hyb}_1 = 1] - \Pr[\text{Hyb}_2 = 1]| \leq \text{Adv}_{\text{PKE}}^{\text{sem}}(\lambda)$, since $\mathcal{A}_2$ receives no information about i_b . By symmetry, $\Pr[\text{Hyb}_2 = 1] = \Pr[\text{Hyb}_3 = 1] = 1/2$. Therefore, $\text{Adv}_{\mathcal{A}}^{s-anon}(\lambda) \leq \text{Adv}_{\text{NIZK}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{PKE}}^{\text{sem}}(\lambda)$. Hence, $\Pi_{PS-NIBS}$ provides strong posterior anonymity.

Now, given adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ against Exp^{s-hide} :

- Hyb₀: Real experiment with $b=0$
- Hyb₁: Replace π_{CS} with simulated proof
- Hyb₂: Swap com with commitment to $0^{|\mu|}$
- Hyb₃: Substitute ct_{CS} with encryption of $0^{|I|}$
- Hyb₄: As Hyb₃ but with $b=1$

Indistinguishability follows from ZKP zero-knowledge (Hyb₀ to Hyb₁), commitment hiding (Hyb₁ to Hyb₂), and PKE semantic security (Hyb₂ to Hyb₃), so, strong posterior message hiding is getting achieved by $\Pi_{PS-NIBS}$.

To check unforgeability, given forger $\mathcal{F}$ outputting (pk^*, σ^*, μ^*) where $\sigma^* = (\pi^*, ct^*)$, by ZKP soundness, if $\text{PS.CVerify}(\sigma^*, \cdot, pk^*) = 1$, then with overwhelming probability:

- For incognito: $\exists i \in [n]$ s.t. $\mathbf{C}_i \in \mathbf{pk}^*$ and valid (σ_i, μ_i) exists
- For concealed: $\exists$ valid (σ, μ) for the committed message

If $\mathcal{F}$ never queried for these values, we construct the rOM-ISIS solver: Extract witness (i^*, σ^*, μ^*) from π^* (by NIZK knowledge soundness), then parse $\mu^* = (\mathbf{w}^*, \delta^*)$, $\sigma^* = (\pi', \mathbf{ct}')$, eventually, if $\mathcal{F}$ never obtained signature on μ^* from $\mathbf{C}_{i^*}$, this breaks one-more-unforgeability of NIBS_{rOM}. Therefore, $\Pr[\text{Exp}^{\text{forge}} = 1] \leq \text{Adv}_{\text{NIBS}}^{\text{omuf}}(\lambda) + \text{Adv}_{\text{NIZK}}^{\text{sound}}(\lambda) + \text{Adv}_{\text{COM}}^{\text{bind}}(\lambda)$. $\square$

5 Post-quantum Blind Vote

We present Post-quantum Blind Vote PQBV in Figure 3, a blockchain-based voting scheme that achieves voter privacy through polynomial commitments and posterior-secure blind signatures.

The election authority EA deploys the PQBV smart contract on the blockchain with the following parameters: PS-NIBS public parameters $\mathbf{pp}_{\text{NIBS}}$, verification key $\mathbf{vk}_S$ for blind signature validation, polynomial commitment key $\mathbf{ck}_{\text{PC}}$, and economic parameters including maximum voters n_{max} , registration fee f , relay reward ρ , deposit δ , and time boundaries $\{t_i\}_{i=1}^6$. The protocol operates in distinct phases: registration, commitment, reveal, and tallying.

In this scheme, votes are encoded as polynomials rather than encrypted. Each voter encodes their vote $v \in \{0, 1\}$ as a degree-1 polynomial $h(X) = v \cdot X^0 + r \cdot X^1$ where r is a random blinding factor. Privacy is achieved through: (1) polynomial commitments that hide the vote, (2) posterior-secure blind signatures that prevent linking voters to ballots, (3) decoy sets that provide anonymity among multiple verification keys, and (4) the commit-reveal mechanism where voters first commit to their vote polynomials and later reveal only evaluations at a blockchain-generated challenge point.

The setup algorithm generates PS-NIBS parameters and signer keypair $(\mathbf{sk}_S, \mathbf{vk}_S)$, initializes polynomial commitment with key $\mathbf{ck}_{\text{PC}}$, deploys the smart contract with economic parameters, and outputs public key $\mathbf{pk} = (\mathbf{pp}_{\text{NIBS}}, \mathbf{ck}_{\text{PC}}, \mathbf{vk}_S)$ and secret key $\mathbf{sk} = \mathbf{sk}_S$.

The registration sub-procedure generates receiver keypair $(\mathbf{sk}_R, \mathbf{pk}_R)$ for the blind signature protocol, submits $h_{\text{reg}} = H(\text{id} \parallel \mathbf{pk}_R)$ to the blockchain with fee f , and outputs credential $c = (\mathbf{sk}_R, \mathbf{pk}_R)$.

The voting function obtains blind signature (μ, σ) through off-chain interaction with authority, encodes vote as polynomial $h(X) = v \cdot X^0 + r \cdot X^1$, creates commitment $(\text{com}_v, \delta_v) \leftarrow \text{PC.Com}(\mathbf{ck}_{\text{PC}}, h(X))$, forms message $\mu_{\text{com}} = (\text{com}_v, H(\text{id}))$, retrieves $k-1$ decoy verification keys from blockchain, converts signature to posterior-secure form $(\sigma_{\text{PS}}, \mathbf{aux}) \leftarrow \Pi_{\text{PS-NIBS}}.\text{PS.Convert}(\sigma, \mu_{\text{com}}, \mathbf{vk}_S, \{\mathbf{vk}_i\}_{i=1}^k)$, commits $h_{\text{com}} = H(\sigma_{\text{PS}})$ on-chain receiving reward ρ , waits for challenge $x = H(\text{block.number} \parallel \text{"challenge"})$, evaluates the polynomial $(y, \rho) \leftarrow \text{PC.Eval}(x, \delta_v)$ where $y = h(x) = v + r \cdot x$, and reveals $(\sigma_{\text{PS}}, \mathbf{aux}, \mu_{\text{com}}, y, \rho)$ on-chain receiving another reward ρ .

Tallying collects all revealed evaluations $\mathcal{Y} = \{y_i\}$ and computes $r = \sum_i y_i \bmod p = \sum_i (v_i + r_i \cdot x) = \sum_i v_i + x \cdot \sum_i r_i$, which reveals the vote sum plus a randomness term dependent on challenge x . The authority generates a zero-knowledge proof

Π demonstrating correct aggregation locally (off-chain) and then submits it to the blockchain contract and publishes (r, Π) .

Verification confirms all commitments have corresponding reveals, validates each reveal's posterior-secure signature, decommitment, and polynomial evaluation at challenge point x , ensuring protocol integrity without revealing individual votes.

Off-chain operations include all NIBS operations except final verification, polynomial creation and evaluation preparation, and signature aggregation for batch submission. As Figure 5 shows, it takes 32 bytes per vote (just the value), and 32 bytes per commitment in comparison with Blind Vote [5], taking ~ 1 KB. Also, PQBV does not need storage of full signatures or proofs. Multiple voters can share one transaction, batch verification of NIBS signatures, and amortized gas costs across voters are other optimizations of PQBV compared to Blind Vote. The challenge has been simplified as well by providing a deterministic challenge from the block hash, and no need to store individual randomness.

6 Privacy and Efficiency Analysis

6.1 Ballot Privacy of PQBV

Having established the construction of our posterior-secure blind signature scheme $\Pi_{PS-NIBS}$ and its integration into the PQBV voting scheme, we now turn to the formal privacy analysis; proceeding with Theorem 2 for showing PQBV satisfies the ballot privacy [23] definition (Definition 3).

Theorem 2. *Let $\Pi_{PS-NIBS}$ be a posterior-secure non-interactive blind signature scheme (Definition 1) that satisfies unforgeability (Theorem 1), let PC be a polynomial commitment scheme (Definition 2) that satisfies hiding and binding properties, and let H be a random oracle. Then the PQBV voting scheme achieves ballot privacy BPRIV according to Definition 3.*

Before proving Theorem 2, we construct the simulator for PQBV as Sim in the Definition 5 in the following, and then prove ballot privacy through a sequence of hybrid games by Lemmas 3, 4, 5, 6.

Definition 5. *The simulator $\text{Sim}(\text{pk}, \text{BB}_1, r)$ takes as input the public key $\text{pk} = (\text{pp}_{\text{NIBS}}, \text{ck}_{\text{PC}}, \text{vk}_S)$, the bulletin board BB_1 from world $\beta = 1$, and the tally result r , and operates as follows: Extract all revealed polynomial evaluations $\mathcal{Y}_1 = \{y_i^{(1)}\}_{i=1}^n$ from $\text{BB}_1.\text{reveals}$, then calculate adjustment $\Delta = r - \sum_{i=1}^n y_i^{(1)} \bmod p$, and generate simulated proof $\Pi' \leftarrow \text{ZK.Sim}(\text{pk}, r, \mathcal{Y}_1, \Delta)$. Finally return Π' as output.*

Proof of Theorem 2. Let $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ be any polynomial-time adversary attacking the ballot privacy of PQBV.

- G_0 : The real ballot privacy experiment $\text{Exp}_{\mathcal{A}, \text{PQB}, \text{Sim}, I}^{\text{BPRIV}, 0}(\lambda)$.
- G_1 : Same as G_0 , but polynomial commitments for honest voters are replaced with commitments to random degree-1 polynomials.

Setup ($1^\lambda, \text{aux}, \text{VoterList}$) 1: $\text{pp}_{\text{NIBS}} \leftarrow \Pi_{\text{PS-NIBS}}.\text{Setup}(1^\lambda)$ 2: $(\text{sk}_S, \text{vk}_S) \leftarrow \Pi_{\text{PS-NIBS}}.\text{KeyGen}_S(\text{pp}_{\text{NIBS}})$ 3: $\text{ck}_{\text{PC}} \leftarrow \text{PC}.\text{Setup}(1^\lambda, \text{VoterList})$ 4: $\text{params} \leftarrow (n_{\text{max}}, f, \rho, \delta, \{t_i\}_{i=1}^6)$ 5: $\text{contract} \leftarrow \text{Blockchain}.\text{Deploy}(\text{pp}_{\text{NIBS}}, \text{vk}_S, \text{ck}_{\text{PC}}, \text{params})$ 6: $\text{pk} \leftarrow (\text{pp}_{\text{NIBS}}, \text{ck}_{\text{PC}}, \text{vk}_S)$ 7: $\text{sk} \leftarrow \text{sk}_S$ 8: return $(\text{pk}, \text{sk}, \text{contract})$	Verify ($\text{id}, \text{state}, \text{BB}$) 1: $(v, h, \delta_v, x, \text{aux}) \leftarrow \text{state}$ 2: $e_1 \leftarrow \exists h_{\text{com}} \in \text{BB}.\text{commitments}$ 3: $e_2 \leftarrow \exists (\sigma_{\text{PS}}, y) \in \text{BB}.\text{reveals} : H(\sigma_{\text{PS}}) = h_{\text{com}}$ 4: $(\text{com}_v, \cdot) \leftarrow \mu_{\text{com}}$ 5: $e_3 \leftarrow \text{PC}.\text{Verify}(\text{ck}_{\text{PC}}, \text{com}_v, x, y, \rho)$ 6: return $(e_1 \wedge e_2 \wedge e_3)$
Register ($1^\lambda, \text{id}, \text{contract}$) 1: $(\text{sk}_R, \text{pk}_R) \leftarrow \Pi_{\text{PS-NIBS}}.\text{KeyGen}_R(\text{pp}_{\text{NIBS}})$ 2: $h_{\text{reg}} \leftarrow H(\text{id} \parallel \text{pk}_R)$ 3: $\text{tx} \leftarrow \text{Blockchain}.\text{Submit}(\text{contract}.\text{register}(h_{\text{reg}}), f)$ 4: $c \leftarrow (\text{sk}_R, \text{pk}_R)$ 5: return c	Valid (BB, pk) 1: $(\text{pp}_{\text{NIBS}}, \text{ck}_{\text{PC}}, \text{vk}_S) \leftarrow \text{pk}$ 2: $e_1 \leftarrow \text{BB}.\text{commitments} = \text{BB}.\text{reveals} $ 3: for $(\sigma_{\text{PS}}, \text{aux}, \mu_{\text{com}}, y, \rho)$ in $\text{BB}.\text{reveals}$ do 4: $(\pi_{\text{PS}}, C_\mu, \{\text{vk}_i\}_{i=1}^k) \leftarrow \sigma_{\text{PS}}$ 5: $(\text{com}_v, h_{\text{id}}) \leftarrow \mu_{\text{com}}$ 6: $e_2 \leftarrow \Pi_{\text{PS-NIBS}}.\text{PS}.\text{CVerify}(\sigma_{\text{PS}}, \{\text{vk}_i\}_{i=1}^k)$ 7: $e_3 \leftarrow \Pi_{\text{PS-NIBS}}.\text{PS}.\text{Decom}(\mu_{\text{com}}, \sigma_{\text{PS}}, \text{aux})$ 8: $x \leftarrow H(\text{BB}.\text{block}.\text{number} \parallel \text{"challenge"})$ 9: $e_4 \leftarrow \text{PC}.\text{Verify}(\text{ck}_{\text{PC}}, \text{com}_v, x, y, \rho)$ 10: if $\neg(e_2 \wedge e_3 \wedge e_4)$ then return $\perp$ 11: return e_1
Vote ($\text{pk}, \text{id}, c, v$) 1: $(\text{pp}_{\text{NIBS}}, \text{ck}_{\text{PC}}, \text{vk}_S) \leftarrow \text{pk}$ 2: $(\text{sk}_R, \text{pk}_R) \leftarrow c$ 3: $(\text{psig}, \text{nonce}) \leftarrow \Pi_{\text{PS-NIBS}}.\text{Issue}(\text{sk}_S, \text{pk}_R)$ 4: $(\mu, \sigma) \leftarrow \Pi_{\text{PS-NIBS}}.\text{Obtain}(\text{sk}_R, \text{vk}_S, \text{psig}, \text{nonce})$ 5: $r \leftarrow \mathbb{Z}_p$ 6: $h(X) \leftarrow v \cdot X^0 + r \cdot X^1 \in \mathbb{Z}_p[X]$ 7: $(\text{com}_v, \delta_v) \leftarrow \text{PC}.\text{Com}(\text{ck}_{\text{PC}}, h(X))$ 8: $\mu_{\text{com}} \leftarrow (\text{com}_v, H(\text{id}))$ 9: $\text{DecoySet} \leftarrow \text{Blockchain}.\text{GetDecoys}(\text{contract}, k-1)$ 10: $\{\text{vk}_i\}_{i=1}^k \leftarrow \{\text{vk}_S\} \cup \text{DecoySet}$ 11: $(\sigma_{\text{PS}}, \text{aux}) \leftarrow \Pi_{\text{PS-NIBS}}.\text{PS}.\text{Convert}(\sigma, \mu_{\text{com}}, \text{vk}_S, \{\text{vk}_i\}_{i=1}^k)$ 12: $h_{\text{com}} \leftarrow H(\sigma_{\text{PS}})$ 13: $\text{Blockchain}.\text{Submit}(\text{contract}.\text{commit}(h_{\text{com}}))$ 14: $x \leftarrow H(\text{block}.\text{number} \parallel \text{"challenge"})$ 15: $(y, \rho) \leftarrow \text{PC}.\text{Eval}(x, \delta_v)$ 16: $\text{Blockchain}.\text{Submit}(\text{contract}.\text{reveal}(\sigma_{\text{PS}}, \text{aux}, \mu_{\text{com}}, y, \rho))$ 17: $p \leftarrow (\mu, \sigma_{\text{PS}})$ 18: $b \leftarrow (\text{com}_v, y)$ 19: $\text{state} \leftarrow (v, h, \delta_v, x, \text{aux})$ 20: return (p, b, state)	Tally (BB, sk) 1: $\mathcal{Y} \leftarrow \emptyset$ 2: for (σ_{PS}, y) in $\text{BB}.\text{reveals}$ do 3: $\mathcal{Y} \leftarrow \mathcal{Y} \cup \{y\}$ 4: $r \leftarrow \sum_{y \in \mathcal{Y}} y \bmod p$ 5: $\Pi \leftarrow \text{ZK}.\text{Prove}(\text{sk}, \mathcal{Y}, r)$ 6: $\text{Blockchain}.\text{Submit}(\text{contract}.\text{publishResult}(r, \Pi))$ 7: return (r, Π)

Figure 3: PQBV with $\Pi_{\text{PS-NIBS}} = (\text{Setup}, \text{KeyGen}_S, \text{KeyGen}_R, \text{Issue}, \text{Obtain}, \text{Verify}, \text{PS}.\text{Convert}, \text{PS}.\text{CVerify}, \text{PS}.\text{Decom})$, $\text{PC} = (\text{Setup}, \text{Com}, \text{Open}, \text{Eval}, \text{Verify})$, and $\text{Blockchain} = (\text{Deploy}, \text{Submit}, \text{GetDecoys})$.

- G_2 : Same as G_1 , but posterior-secure blind signatures for honest voters are replaced with simulated unlinkable signatures.
- G_3 : Same as G_2 , but polynomial evaluations are replaced with uniformly random values from $\mathbb{Z}_p$.
- G_4 : The simulated ballot privacy experiment $\text{Exp}_{\mathcal{A}, \text{PQBV}, \text{Sim}, I}^{\text{BPRIV}, 1}(\lambda)$.

Lemma 3. $|\Pr[G_0 = 1] - \Pr[G_1 = 1]| \leq \text{negl}(\lambda)$

Proof of Lemma 3. We construct a reduction $\mathcal{B}_1$ that uses any distinguisher between G_0 and G_1 to break the hiding property of the polynomial commitment scheme PC. Given commitment hiding challenger with public parameters ck_{PC} , $\mathcal{B}_1$ proceeds as follows:

For each honest voter $i \in I$, $\mathcal{B}_1$ defines two polynomials $h_0^{(i)}(X) = v_i^{(0)} \cdot X^0 + r_i \cdot X^1$ (real vote polynomial) and $h_1^{(i)}(X) = s_i \cdot X^0 + t_i \cdot X^1$ (random polynomial), where $v_i^{(0)}$ is the vote in world $\beta = 0$, $r_i \leftarrow \mathbb{Z}_p$ is the blinding factor, and $s_i, t_i \leftarrow \mathbb{Z}_p$ are uniformly random. $\mathcal{B}_1$ sends $(h_0^{(i)}, h_1^{(i)})$ to the hiding challenger and receives commitment com_i^* . $\mathcal{B}_1$ embeds com_i^* as the polynomial commitment for voter i in the ballot privacy experiment. $\mathcal{B}_1$ simulates all other protocol components honestly, including the posterior-secure blind signature generation and the reveal phase. When $\mathcal{A}_2$ outputs a bit β' , $\mathcal{B}_1$ outputs β' .

If the challenger commits to $h_0^{(i)}$, we have game G_0 . If the challenger commits to $h_1^{(i)}$, we have game G_1 . Therefore $|\Pr[\mathcal{B}_1 = 1 | \beta = 0] - \Pr[\mathcal{B}_1 = 1 | \beta = 1]| = |\Pr[G_0 = 1] - \Pr[G_1 = 1]|$; by the hiding property of PC, this advantage is negligible. $\square$

Lemma 4. $|\Pr[G_1 = 1] - \Pr[G_2 = 1]| \leq \text{negl}(\lambda)$

Proof of Lemma 4. We construct a reduction $\mathcal{B}_2$ that uses any distinguisher between G_1 and G_2 to break the unlinkability property of $\Pi_{\text{PS-NIBS}}$. The unlinkability provides $\mathcal{B}_2$ with public parameters pp_{NIBS} and signer verification key vk_S , access to a signing oracle that can produce blind signatures, and a challenge consisting of signatures that are either real or unlinkable. $\mathcal{B}_2$ proceeds as follows:

For each honest voter i , $\mathcal{B}_2$ generates receiver keypair $(\text{sk}_R^{(i)}, \text{pk}_R^{(i)}) \leftarrow \Pi_{\text{PS-NIBS}}.\text{KeyGen}_R(\text{pp}_{\text{NIBS}})$. $\mathcal{B}_2$ queries the signing oracle to obtain blind signatures (μ_i, σ_i) for messages $\mu_{\text{com}}^{(i)} = (\text{com}_i, H(\text{id}_i))$. For the posterior-secure conversion, $\mathcal{B}_2$ generates decoy verification keys $\{\text{vk}_j\}_{j=2}^k$ and sets $\{\text{vk}_j\}_{j=1}^k = \{\text{vk}_S\} \cup \{\text{vk}_j\}_{j=2}^k$. $\mathcal{B}_2$ computes the posterior-secure signatures:

$$(\sigma_{\text{PS}}^{(i)}, \text{aux}^{(i)}) \leftarrow \Pi_{\text{PS-NIBS}}.\text{PS.Convert}(\sigma_i, \mu_{\text{com}}^{(i)}, \text{vk}_S, \{\text{vk}_j\}_{j=1}^k)$$

The key insight is that the posterior-secure conversion with $k-1$ decoy keys makes it computationally infeasible to link $\sigma_{\text{PS}}^{(i)}$ to voter i , even given sk_S . $\mathcal{B}_2$ embeds the challenge signatures in the ballot privacy experiment and outputs whatever $\mathcal{A}_2$ outputs.

The unlinkability property ensures that:

$$|\Pr[\mathcal{B}_2 = 1 | \text{real signatures}] - \Pr[\mathcal{B}_2 = 1 | \text{unlinkable signatures}]| \leq \text{negl}(\lambda)$$

This directly translates to the bound in the lemma statement. $\square$

Lemma 5. $|\Pr[G_2 = 1] - \Pr[G_3 = 1]| \leq \text{negl}(\lambda)$

Proof of Lemma 5. For each honest voter i , the revealed value in G_2 is $y_i = h_i(x) = v_i + r_i \cdot x$, where $h_i(X) = v_i \cdot X^0 + r_i \cdot X^1$ is a random degree-1 polynomial (by the modification in G_1), $r_i \leftarrow \mathbb{Z}_p$, and $x = H(\text{block.number} || \text{"challenge"})$. In this case, there are

some key observations: I) The challenge point x is chosen *after* the commitment phase via the random oracle H , making it unpredictable during commitment. II) The blinding factor r_i is chosen uniformly at random and independently for each voter. III) The term $r_i \cdot x$ is uniformly distributed in $\mathbb{Z}_p$ since r_i is uniform and x is independent of r_i .

Therefore, for any fixed vote value v_i , the distribution of $y_i = v_i + r_i \cdot x$ is uniform over $\mathbb{Z}_p$. In G_3 , we replace each y_i with a uniformly random value $y'_i \leftarrow \mathbb{Z}_p$. Since both distributions are uniform over $\mathbb{Z}_p$, they are statistically identical $|\Pr[G_2=1] - \Pr[G_3=1]| = 0$. $\square$

Lemma 6. $|\Pr[G_3=1] - \Pr[G_4=1]| \leq \text{negl}(\lambda)$

Proof of Lemma 6. This follows from the zero-knowledge property of the aggregation proof system. In G_3 , the tally oracle computes $r = \sum_{i=1}^n y_i \bmod p$ ($y_i \sim \mathcal{U}(a, b)$) and $\Pi \leftarrow \text{ZK.Prove}(\text{sk}; \text{correct aggregation of } \{y_i\})$.

In G_4 , the tally oracle uses the simulator for r = same result as in G_3 as $Pi' \leftarrow \text{Sim}(\text{pk}, \text{BB}_1, r)$. The simulator Sim works by computing the difference $\Delta = r - \sum_j y_j^{(1)} \bmod p$ where $\{y_j^{(1)}\}$ are the values from world $\beta=1$, and using the zero-knowledge simulator: $\Pi' \leftarrow \text{ZK.Sim}(\text{pk}, r, \{y_j^{(1)}\}, \Delta)$. By the zero-knowledge property:

$$\{\Pi : \Pi \leftarrow \text{ZK.Prove}(\text{sk}; \text{statement})\} \approx_c \{\Pi' : \Pi' \leftarrow \text{ZK.Sim}(\text{pk}, \text{statement})\}$$

Therefore, $|\Pr[G_3=1] - \Pr[G_4=1]| \leq \text{negl}(\lambda)$. $\square$

Combining all Lemmas 3, 4, 5, 6 we have the final bound:

$$\begin{aligned} & \left| \Pr[\text{Exp}_{\mathcal{A}, \text{PQBV}, \text{Sim}, I}^{\text{BPRIV}, 0}(\lambda) = 1] - \Pr[\text{Exp}_{\mathcal{A}, \text{PQBV}, \text{Sim}, I}^{\text{BPRIV}, 1}(\lambda) = 1] \right| \\ &= |\Pr[G_0=1] - \Pr[G_4=1]| \\ &\leq \sum_{i=0}^3 |\Pr[G_i=1] - \Pr[G_{i+1}=1]| \leq 4 \cdot \text{negl}(\lambda) = \text{negl}(\lambda) \end{aligned}$$

Therefore, PQBV achieves ballot privacy BPRIV according to Definition 3. $\square$

6.2 Efficiency Analysis of PQBV

In [5], it has been demonstrated that Blind Vote outperforms other known economical blockchain-based e-voting schemes, including Boardroom Voting [24], Tornado Vote [3], and Metamask [25]. Here, we examine the post-quantum variant of Blind Vote, and among other schemes, Epoque [14] is the most similar to the structure of PQBV. As a result, PQBV achieves better efficiency than Epoque in the tally phase due to eliminating homomorphic decryption, while maintaining comparable or better performance in other phases.

In the Setup phase of PQBV, the PS-NIBS signer key generation needs $O(m^2 \cdot \text{poly}(\lambda))$ for lattice trapdoor generation, polynomial commitment setup $O(N \cdot \text{poly}(\lambda))$ where N is the degree bound. Then, in the Voting phase PS-NIBS Issue/Obtain takes $O(m^2 \cdot \text{poly}(\lambda))$, polynomial commitment for degree-1 polynomial $O(\text{poly}(\lambda))$,

	Setup Phase	Vote Phase	Tally Phase	Verification Phase
PQBV (Ours)	$O(m^2 \cdot \text{poly}(\lambda) + N \cdot \text{poly}(\lambda))$	$O(k \cdot m^2 \cdot \text{poly}(\lambda))$	$O(n \cdot \text{poly}(\lambda))$	$O(n \cdot k \cdot m^2 \cdot \text{poly}(\lambda))$
Epoque [14]	$O(n \cdot \mathfrak{t} \cdot m^3 \cdot \log^2 q)$	$O(\mathfrak{v} \cdot \mathfrak{c} \cdot \mathfrak{t} \cdot m \cdot n \cdot \log^2 q)$	$O(\mathfrak{v} \cdot \mathfrak{t} \cdot m \cdot n \cdot \log^2 q)$	$O(\mathfrak{v} + \mathfrak{c} \cdot \mathfrak{t})$

Table 1: Cumulative time complexity of schemes in each phase, assuming $\mathfrak{c}$, $\mathfrak{t}$, and $\mathfrak{v}$ be the number of candidates, trustees, and voters, respectively, in **Epoque**.

PS.Convert with k decoys needs $O(k \cdot m^2 \cdot \text{poly}(\lambda))$, on-chain commit $O(1)$, PC.Eval takes $O(\text{poly}(\lambda))$, and on-chain reveal with PS.CVerify and PS.Decom verification $O(k \cdot m^2 \cdot \text{poly}(\lambda))$. In Tally, collecting n revealed vote values takes $O(n)$, aggregating by simple summation $O(n)$, generating proof for correct aggregation $O(n \cdot \text{poly}(\lambda))$, and publishing result $O(1)$. Finally, in global verification, it verifies all n reveals with PS.CVerify at cost $O(k \cdot m^2 \cdot \text{poly}(\lambda))$ each.

7 Conclusion

In this work, we took a step toward bridging the gap between cryptographic security requirements and the cryptocurrency community’s aspirations for blockchain-based voting by presenting PQBV, the first provably secure post-quantum blockchain voting system, along with addressing quantum threats through lattice-based primitives. The integration of posterior-secure blind signatures enables enhanced privacy guarantees, allowing for both voter anonymity and message hiding capabilities that can be applied retroactively to already-generated signatures. Through our analysis, we showed that PQBV offers promising efficiency compared to other schemes, such as **Epoque**, while maintaining formal security proofs for ballot privacy. As future directions, we aim for the realistic implementation of PQBV for demonstrating feasibility and discussing practical deployment.

8 Acknowledgment

N. Abapour was funded by the Computer Science Research Centre (Grant No. AB8031) and FEPS (Grant No. TB8071), both from UniOfSurrey. C. C. Drăgan is partially supported by TrustVote – EPSRC grant EP/Y020529/1, AP4L - EPSRC grant EP/W032473/1, CONNECT - Horizon Europe Guarantee 10043730 and EU Horizon grants 101069688, REWIRE - Horizon Europe Guarantee 10043743 and EU Horizon grants 101070627. M. Mahdavi was supported by the Spanish Ministry of Science and Innovation through the PID2021-125962OB-C31 “SECURING” project, along with the ARTEMISA International Chair of Cybersecurity (C057/23) and the DANGER Strategic Project of Cybersecurity (C062/23), both funded by the Spanish National Institute of Cybersecurity through the European Union — NextGenerationEU and the Recovery, Transformation, and Resilience Plan.

A Appendix

A.1 On-chain Operations of Smart Contract

According to Figure 5 and Figure 4, the smart contract operations enforce timing constraints, verify posterior-secure signatures using $\Pi_{\text{PS-NIBS}}.\text{PS.CVerify}$, validate decommitments via $\Pi_{\text{PS-NIBS}}.\text{PS.Decom}$, and confirm polynomial evaluations through PC.Verify .

A.2 Concealed and Incognito Signatures

As Figure 6 shows, the incognito signature transformation enhances blind signatures with posterior anonymity by encrypting the signer's identity with signature components. A verifier receiving (π_I, ct_I) can confirm that some authorized signer from VK produced a valid signature without learning the specific signer. This leverages encryption, semantic security, and NIZK zero-knowledge to keep the encrypted identity computationally hidden, while π_I proves the ciphertext contains a valid signature from an authorized party.

```
1 pragma solidity ^0.8.19;
2 contract PosteriorSignatureCommitment {
3     // commitment
4     struct PosteriorCommitment {
5         bytes32 sigHash; // H(\sigma_{PS})
6         uint256 timestamp;
7         bool revealed;
8     }
9     // commitment storage
10    mapping(bytes32 => PosteriorCommitment) public commitments;
11    // protocol parameters
12    uint256 public t_commit_end;
13    uint256 public relay_reward;
14    // events
15    event CommitmentSubmitted
16        (bytes32 indexed sigHash);
17    // batch commit
18    // posterior signatures with relay incentive
19    function batchCommitPS(bytes[]
20        memory signatures_PS) external payable {
21        require(
22            (block.timestamp <= t_commit_end
23            , "Commitment period ended");
24        for (uint i = 0; i < signatures_PS.length; i++) {
25            bytes32
26            h = keccak256(signatures_PS[i]);
27            require(commitments[h].timestamp
28            == 0, "Duplicate commitment");
29            commitments
30            [h] = PosteriorCommitment({
31                sigHash: h,
32                timestamp: block.timestamp,
33                revealed: false
34            });
35            emit CommitmentSubmitted(h);
36        }
37        // single
38        relay payment for batch submission
39        payable(msg.sender).transfer(
40            relay_reward * signatures_PS.length);
41    }
42 }
```

```
1 pragma solidity ^0.8.19;
2 contract PosteriorSignatureReveal {
3     // reveal function for
4     // posterior signatures with vote extraction
5     function revealPS(
6         bytes memory sigma_PS,
7         bytes memory aux,
8         uint256 com_vote,
9         bytes32 voter_hash,
10        uint256 y,
11        bytes memory rho
12    ) external {
13        require(block.timestamp >=
14            t_reveal_start, "Reveal not started");
15        bytes32 h = keccak256(sigma_PS);
16        PosteriorCommitment
17        storage pc = commitments[h];
18        require(pc.timestamp > 0
19            && !pc.revealed, "Invalid commitment");
20        // Parse \sigma_{PS} components
21        (bytes memory pi_PS, bytes32
22        C_mu, address[] memory decoys) =
23        parsePosteriorSig(sigma_PS);
24        // Verify posterior-secure signature
25        require(PS_NIBS.PS_CVerify
26            (sigma_PS, decoys), "Invalid PS sig");
27        // Decommit message
28        bytes memory mu_com
29        = abi.encode(com_vote, voter_hash);
30        require(PS_NIBS.PS_Decom(mu_com
31            , sigma_PS, aux), "Decommit failed");
32        // Derive and verify challenge
33        uint256 x = uint256(keccak256(abi
34            .encode(block.number, "challenge")));
35        require(PC.verify(ck, com_vote
36            , x, y, rho), "Invalid eval");
37        // Mark as revealed and store vote
38        pc.revealed = true;
39        votes.push(y);
40        emit VoteRevealed(h, y);
41    }
42 }
```

Figure 5: Posterior Signature Reveal

Figure 4: Signature Commitment

IS.Convert(σ, μ, VK)

```

1:  ( $\pi, \text{ct}$ )  $\leftarrow \sigma$ 
2:  ( $\mathbf{w}, \delta$ )  $\leftarrow \mu$ 
3:   $\{\mathbf{C}_1, \dots, \mathbf{C}_n\} \leftarrow \text{VK}$ 
4:   $i^* \leftarrow \{i \in [n] : \text{Verify}(\mathbf{C}_i, \mu, \sigma) = 1\}$ 
5:   $\text{ct}_I \leftarrow \text{PKE.Enc}(\text{pke.pk}, i^* \parallel \sigma \parallel \mu)$ 
6:   $\mathcal{L}_{\text{anon}} := \{(\text{VK}, \text{ct}_I) : \exists(i, \sigma, \mu) \text{ s.t.}$ 
     $\text{Verify}(\mathbf{C}_i, \mu, \sigma) = 1 \wedge \text{ct}_I = \text{PKE.Enc}(\text{pke.pk}, i \parallel \sigma \parallel \mu)\}$ 
7:   $\pi_I \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_{\text{anon}}, (\text{VK}, \text{ct}_I), (i^*, \sigma, \mu))$ 
8:   $\sigma_I := (\pi_I, \text{ct}_I)$ 
9:  return  $\sigma_I$ 

```

IS.CVerify($\sigma_I, \mu', \text{VK}$)

```

1:  ( $\pi_I, \text{ct}_I$ )  $\leftarrow \sigma_I$ 
2:  ( $\mathbf{w}', \delta'$ )  $\leftarrow \mu'$ 
3:  if  $\mathbf{w}' \neq \mathbf{w} \vee \delta' \neq \delta$ :
4:    return 0
5:  return  $\text{NIZK.Verify}(\text{nizk.crs}_{\text{anon}}, (\text{VK}, \text{ct}_I), \pi_I)$ 

```

Figure 6: Incognito Signature (Posterior Anonymity)

Unlike the incognito variant that hides the signer's identity, concealed signature transformation (Figure 7) focuses on hiding the signed message content while maintaining verifiability. The commitment com serves as a public binding to the hidden message, and the zero-knowledge proof π_{CS} demonstrates that the encrypted data contains a valid signature on the committed message. The decommitment CS.Decom allows authorized parties to reveal the message.

The scheme $\Pi_{\text{PS-NIBS}}$ is a single unified construction that supports both posterior security properties through the mode parameter. The party can generate standard blind signatures using original algorithms, convert them to incognito signatures using $\text{PS.Convert}(\cdot, \cdot, \text{anon}, \cdot)$, and convert them to concealed signatures using $\text{PS.Convert}(\cdot, \cdot, \text{hide}, \cdot)$. This modular design allows the same base signature to achieve different posterior security properties as needed. We investigate the security of $\Pi_{\text{PS-NIBS}}$ in Section 6.

CS.Convert($\sigma, \mu, \mathbf{C}$)

```

1:  ( $\pi, \text{ct}$ )  $\leftarrow \sigma$ 
2:  ( $\mathbf{w}, \delta$ )  $\leftarrow \mu$ 
3:  ( $\text{com}, r$ )  $\leftarrow \text{Com}(\mu)$ 
4:   $\text{ct}_{\text{CS}} \leftarrow \text{PKE.Enc}(\text{pke.pk}, \sigma \parallel \mu \parallel r)$ 
5:   $\mathcal{L}_{\text{hide}} := \{(\mathbf{C}, \text{com}, \text{ct}_{\text{CS}}) : \exists (\sigma, \mu, r) \text{ s.t.}$ 
     $\text{Verify}(\mathbf{C}, \mu, \sigma) = 1 \wedge \text{com} = \text{Com}(\mu; r)\}$ 
6:   $\pi_{\text{CS}} \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_{\text{hide}}, (\mathbf{C}, \text{com}, \text{ct}_{\text{CS}}), (\sigma, \mu, r))$ 
7:   $\sigma_{\text{CS}} := (\pi_{\text{CS}}, \text{ct}_{\text{CS}}, \text{com})$ 
8:   $\text{aux}_{\text{CS}} := (\mu, r)$ 
9:  return ( $\sigma_{\text{CS}}, \text{aux}_{\text{CS}}$ )

```

CS.CVerify($\sigma_{\text{CS}}, \mathbf{C}$)

```

1:  ( $\pi_{\text{CS}}, \text{ct}_{\text{CS}}, \text{com}$ )  $\leftarrow \sigma_{\text{CS}}$ 
2:  return  $\text{NIZK.Verify}(\text{nizk.crs}_{\text{hide}}, (\mathbf{C}, \text{com}, \text{ct}_{\text{CS}}), \pi_{\text{CS}})$ 

```

CS.Decom($\mu', \sigma_{\text{CS}}, \text{aux}_{\text{CS}}$)

```

1:  ( $\mu, r$ )  $\leftarrow \text{aux}_{\text{CS}}$ 
2:  ( $\pi_{\text{CS}}, \text{ct}_{\text{CS}}, \text{com}$ )  $\leftarrow \sigma_{\text{CS}}$ 
3:  return ( $\mu' = \mu \wedge (\text{Decom}(\text{com}, \mu, r) = 1)$ )

```

Figure 7: Concealed Signature (Posterior Message Hiding)

References

- [1] Josh Benaloh, Matthew Bernhard, J. Alex Halderman, Ronald L. Rivest, Peter Y. A. Ryan, Philip B. Stark, Vanessa Teague, Poorvi L. Vora, and Dan S. Wallach. Public evidence from secret ballots. *CoRR*, abs/1707.08619, 2017.
- [2] Rihab H Sahib and Eman S. Al-Shamery. A review on distributed blockchain technology for e-voting systems. *Journal of Physics: Conference Series*, 1804(1):012050, feb 2021.
- [3] Robert Muth and Florian Tschorsch. Tornado vote: Anonymous blockchain-based voting. In *ICBC*, pages 1–9, 2023.
- [4] Noemi Glaeser, István András Seres, Michael Zhu, and Joseph Bonneau. Cicada: A framework for private non-interactive on-chain auctions and voting.
- [5] Amir Kafshdar Goharshady and Zhaorun Lin. Blind vote: Economical and secret blockchain-based voting. In *IEEE Blockchain*, pages 46–53, 2024.

- [6] Patrick Mccorry, Maryam Mehrnezhad, Ehsan Toreini, Siamak F. Shahandashti, and Feng Hao. On secure e-voting over blockchain. *Digital Threats*, 2(4), October 2021.
- [7] *Securing the Vote: Protecting American Democracy*. National Academies Press, August 2018.
- [8] MIT News. Mit researchers identify security vulnerabilities in voting app. <https://news.mit.edu/2020/voting-voatz-app-hack-issues-0213>, 2020. Accessed: 2025-01-26.
- [9] U.S. Vote Foundation. The blockchain threat to democracy. <https://www.usvotefoundation.org/blockchain-threat-to-democracy>. Accessed: 2025-01-26.
- [10] Mahdi Mahdavi, Navid Abapour, and Zahra Ahmadian. Trustworthy approaches to RSA: Efficient exploitation strategies based on common modulus. Cryptology ePrint Archive, Paper 2024/1903, 2024.
- [11] Dustin Moody, Ray Perlner, Andrew Regenscheid, Angela Robinson, and David Cooper. Transition to post-quantum cryptography standards. NIST Internal Report 8547, National Institute of Standards and Technology, November 2024. Initial Public Draft.
- [12] Craig Gidney and Sophie Schmieg. Tracking the cost of quantum factoring. Google Security Blog, May 2025.
- [13] Craig Gidney. How to factor 2048 bit rsa integers with less than a million noisy qubits, 2025.
- [14] Xavier Boyen, Thomas Haines, and Johannes Müller. Epoque: Practical end-to-end verifiable post-quantum-secure e-voting. In *EuroS&P*, pages 272–291, 2021.
- [15] Patrick Hough, Caroline Sandsbråten, and Tjerand Silde. More efficient lattice-based electronic voting from NTRU. *IACR Communications in Cryptology*, 1(4), 2025.
- [16] Guillaume Kaim, Sébastien Canard, Adeline Roux-Langlois, and Jacques Traoré. Post-quantum Online Voting Scheme. In *FC 2021 - Financial Cryptography and Data Security. International Workshops*, volume Lecture Notes in Computer Science, pages 290–305, Virtual event, France, March 2021.
- [17] Tsz Hon Yuen, Ying-Teng Chen, Shimin Pan, Jiangshan Yu, and Joseph K. Liu. Posterior security: Anonymity and message hiding of standard signatures. Cryptology ePrint Archive, Paper 2025/855, 2025.
- [18] Shweta Agrawal, Elena Kirshanova, Damien Stehlé, and Anshu Yadav. Practical, round-optimal lattice-based blind signatures. In *CCS, CCS ’22*, page 39–53, New York, NY, USA, 2022. Association for Computing Machinery.

- [19] Angèle Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75:565–599, 2015.
- [20] Foteini Baldimtsi, Justin Cheng, Rachit Goyal, and Ayush Yadav. Non-interactive blind signatures: Post-quantum and stronger security. In Kai-Min Chung and Yuichi Sasaki, editors, *ASIACRYPT*, volume 15485 of *Lecture Notes in Computer Science*. Springer, Singapore, 2025.
- [21] Intak Hwang, Jinyeong Seo, and Yongsoo Song. Concretely efficient lattice-based polynomial commitment from standard assumptions. In *CRYPTO*, page 414–448, Berlin, Heidelberg, 2024. Springer-Verlag.
- [22] M. Ajtai. Generating hard instances of lattice problems (extended abstract). In *STOC*, STOC '96, page 99–108, New York, NY, USA, 1996. Association for Computing Machinery.
- [23] David Bernhard, Véronique Cortier, David Galindo, Olivier Pereira, and Bogdan Warinschi. Sok: A comprehensive analysis of game-based ballot privacy definitions. In *SECP*, pages 499–516, 2015.
- [24] Patrick McCorry, Siamak F. Shahandashti, and Fang Hao. A smart contract for boardroom voting with maximum voter privacy. In Aggelos Kiayias, editor, *FC*, volume 10322 of *Lecture Notes in Computer Science*. Springer, Cham, 2017.
- [25] Deni Pramulia and Bayu Anggorojati. Implementation and evaluation of blockchain based e-voting system with ethereum and metamask. In *ICIMCIS*, pages 18–23, 2020.